

CONINT Exercise 5

- Take screenshots of the sections where you integrate the is-post-creation-enabled toggle

```
!function(t,e){var o,n,p,r;e._SV||(window.posthog=e,e._i=[],e.init=function(i,s,a){function g(t,e){const n={};let r=i[t];if("object"==typeof r&&null!==r)for(let i in r)n[i]=r[i];n.sessionId=e;return n}(t,s),posthog.init('phc_scuT8Ch2g2hNax2eJMyhipRNEwvUN5sVebwrnD6eU56b',{api_host:'https://eu.i.posthog.com',person_profiles:'identified_only'// or 'always' to create profiles for anonymous users as well}));const posthog_session_id=posthog.get_session_id();posthog.onFeatureFlags(function(){// feature flags should be available at this pointif(posthog.isFeatureEnabled('is-post-creation-enabled')){postForm.style.display='block'}else{postForm.style.display='none'}});if(posthog.getFeatureFlag('experiment-user-button-style')==='test'){userButton.style.backgroundColor='blue';// default style}else{userButton.style.backgroundColor='green'}}});
```

- Capture the code responsible for tracking the creation of a new User and Post in the Fastify Backend

```
const posthog = new PostHog(process.env.POSTHOG_API_KEY!, {
  host: process.env.POSTHOG_HOST,
  enableExceptionAutocapture: true,
});

function getDistinctId(request: FastifyRequest): string {
  const header = request.headers["X-POSTHOG-DISTINCT-ID"];
  if (typeof header === "string" && header) return header;
  return request.ip || "anonymous";
}

function getSessionId(request: FastifyRequest): string {
  const header = request.headers["X-POSTHOG-SESSION-ID"];
  if (typeof header === "string" && header) return header;
  return request.ip || "anonymous";
}

function trackPosthog(
  request: FastifyRequest,
  event: string,
  properties: Record<string | number, any> | undefined,
```

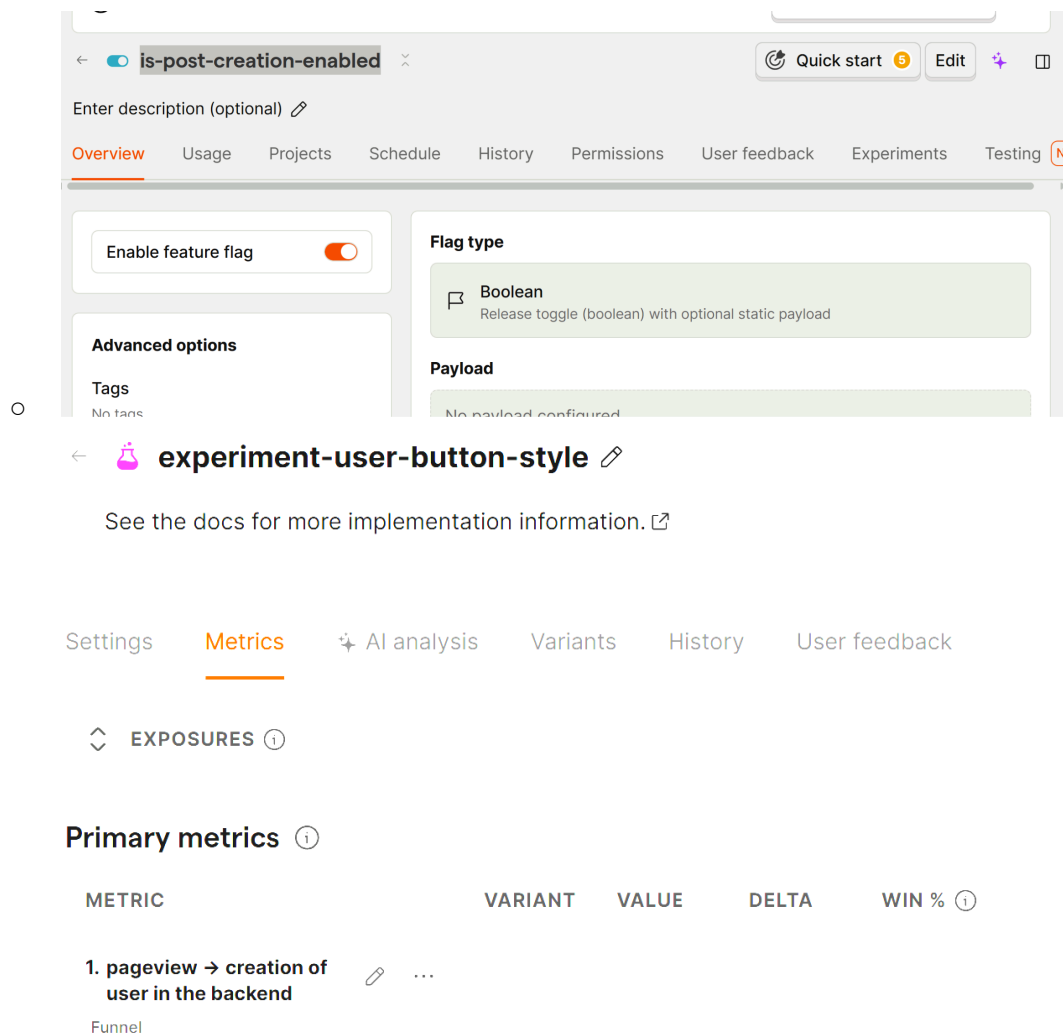
```
    sendFeatureFlags?: boolean,  
  ) {  
    posthog.capture({  
      distinctId: getDistinctId(request),  
      event: event,  
      properties: {  
        '$sessionId': getSessionId(request),  
        ...properties  
      },  
      sendFeatureFlags: sendFeatureFlags,  
    });  
  }  
}
```

- Capture the new Frontend Code for creating users

```
71 // Create a new user (POST)  
72 async function createUser(formData) {  
73   try {  
74     const response = await fetch(USER_API_BASE, {  
75       method: 'POST',  
76       headers: {  
77         'Content-Type': 'application/json',  
78         'X-POSTHOG-DISTINCT-ID': currentUser || posthog.get_distinct_id(), //  
79         'X-POSTHOG-SESSION-ID': posthog_session_id  
80       },  
81       body: JSON.stringify(formData)  
82     });  
83     if (!response.ok) {  
84       const { error } = await response.json();  
85       alert(error);  
86     }  
87     const userData = await response.json();  
88     currentUser = userData.user.id; // Store the created user  
89     localStorage.setItem('currentUser', currentUser); // Store user in localStorag  
90   } catch (error) {  
91     console.error('Error creating user:', error);  
92   }  
93 }
```

```
135 // Handle form submission to create a new user  
136 // the events are tracked by default and categorized by variants  
137 userForm.addEventListener('submit', event => {  
138   event.preventDefault();  
139   const name = userNameInput.value;  
140   const email = userEmailInput.value;  
141   if (name && email) {  
142     createUser({ name, email });  
143   }  
144 });  
145
```

- Include any setup or configuration changes for the PostHog



- Show how the experiment differentiates the UI by applying different background colors (blue vs. green)

```
if (posthog.getFeatureFlag('experiment-user-button-style') === 'test') {  
  userButton.style.backgroundColor = 'blue'; // default style  
} else {  
  userButton.style.backgroundColor = 'green';  
}  
});
```

- include a screenshot that shows a user session assigned to the test group (blue) versus the control group (green).

User Login/Register

Current User ID: 2

Post something

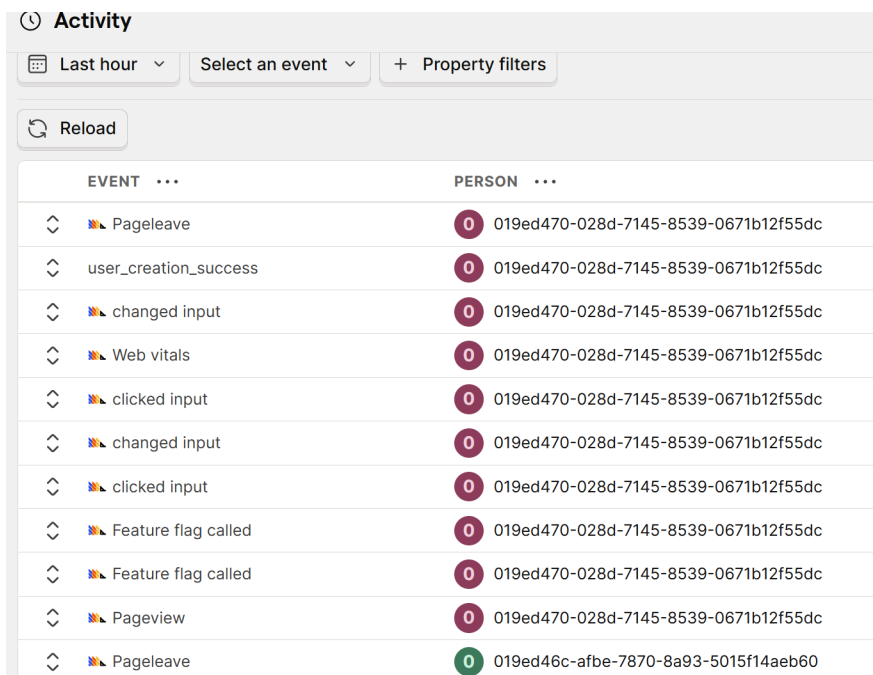
○

User Login/Register

No user logged in.

Post something

- Take screenshots of your PostHog dashboard displaying the events for Post and User creation.

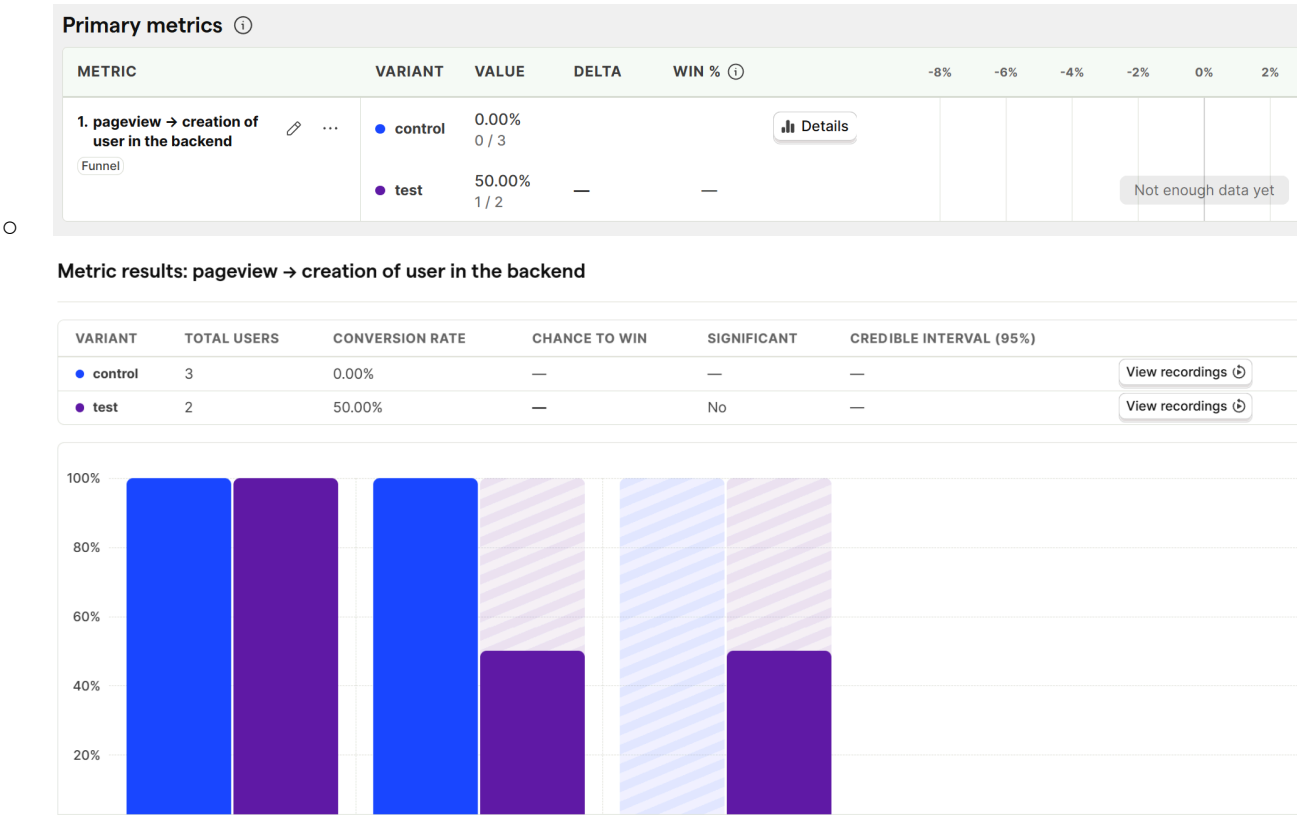


The screenshot shows the PostHog Activity dashboard. At the top, there's a header with a clock icon, the word "Activity", and a dropdown menu set to "Last hour". Below this are buttons for "Select an event", "+ Property filters", and a "Reload" button. The main content is a table with two columns: "EVENT" and "PERSON". The "EVENT" column lists various user interactions, and the "PERSON" column shows the corresponding user ID for each event.

EVENT ...	PERSON ...
🚩 Pageleave	0 019ed470-028d-7145-8539-0671b12f55dc
🚩 user_creation_success	0 019ed470-028d-7145-8539-0671b12f55dc
🚩 changed input	0 019ed470-028d-7145-8539-0671b12f55dc
🚩 Web vitals	0 019ed470-028d-7145-8539-0671b12f55dc
🚩 clicked input	0 019ed470-028d-7145-8539-0671b12f55dc
🚩 changed input	0 019ed470-028d-7145-8539-0671b12f55dc
🚩 clicked input	0 019ed470-028d-7145-8539-0671b12f55dc
🚩 Feature flag called	0 019ed470-028d-7145-8539-0671b12f55dc
🚩 Feature flag called	0 019ed470-028d-7145-8539-0671b12f55dc
🚩 Pageview	0 019ed470-028d-7145-8539-0671b12f55dc
🚩 Pageleave	0 019ed46c-afbe-7870-8a93-5015f14aeb60

○

- Capture the funnel view that tracks the primary metric - from the initial pageview to the successful creation of a User



Git Repository: <https://github.com/if25b190/evaluated-exercise-11>